Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

LowMonitor: Diagnóstico y Monitorización de Hardware

Reporte técnico

Equipo de desarrollo:

Yesfer Bany Fuentes Orozco

0900-17-7256

Astrid Noemí Herrera Solórzano

3490-18-22721

Fatima Paola Ranero García

1490-21-16803

Gilma Odalis Velasquez Pérez

1690-20-25740

Curso: Arquitectura de Computadoras 2

Docente: Ing. René Ornélyz

Guatemala, octubre del 2026

Resumen

LowMonitor es una aplicación de línea de comandos desarrollada en ensamblador NASM para sistemas Linux x86-64. El código fuente organiza seis módulos para identificar el procesador, inspeccionar puertos de entrada/salida, probar un bloque de memoria, medir rendimiento del CPU, estimar latencias de la jerarquía de memoria y ejecutar operaciones SIMD. El proyecto evita enlazar la biblioteca estándar de C y utiliza llamadas al sistema de Linux, con compilación automatizada mediante Makefile y GNU ld. El repositorio también incorpora pruebas de humo y flujos de integración continua. Este reporte documenta la implementación visible en el código y el diseño de pruebas, se presenta como verificados resultados de ejecución.

Introducción

El propósito de LowMonitor es aplicar conceptos de arquitectura de computadoras mediante rutinas de bajo nivel que consultan capacidades del procesador, interactúan con recursos de E/S y observan el comportamiento de memoria y rendimiento. El repositorio implementa una arquitectura modular y documenta las decisiones tomadas para equilibrar el uso de instrucciones nativas, la seguridad y la reproducibilidad.

Objetivos

Objetivo general

Describir la organización técnica de LowMonitor, sus módulos y mecanismos de validación, con base en el código fuente y la documentación entregada.

Objetivos específicos

Caracterizar las funciones implementadas; explicar las decisiones de arquitectura y manejo de privilegios; y resumir las pruebas automatizadas.

Arquitectura e implementación

El punto de entrada está en *src/main.asm*; las rutinas se distribuyen entre *src/mod/*, *src/lib/* y *src/inc/*. El Makefile ensambla los archivos como objetos ELF64 con NASM y los enlaza con GNU ld. Según el registro de decisiones, el programa no enlaza libc: las operaciones de salida, memoria, señales y tiempo se realizan mediante llamadas al sistema, y una biblioteca propia convierte números y métricas a texto. Las funciones internas siguen la convención System V AMD64.

M1. Identificación del CPU

El módulo *cpu.asm* utiliza CPUID para obtener fabricante, nombre/modelo y características del procesador, y contempla extensiones SSE, SSE2, AVX, AVX2 y AVX-512. Para las instrucciones vectoriales, el proyecto también comprueba el soporte del sistema operativo mediante XCR0, de modo que la selección de una ruta SIMD depende tanto de la CPU como del estado habilitado por el sistema.

¿Qué es y qué hace? Es el módulo encargado de identificar las características principales del procesador (CPU). Obtiene información sobre el fabricante, modelo, arquitectura, cantidad de hilos lógicos y las instrucciones que soporta el procesador.

¿Cómo lo hace? Consulta las instrucciones CPUID del procesador, que permiten obtener información sobre sus características y capacidades.

```
  [PASA]  resumen: throughput y benchmark
  [PASA]  escritura normal vs non-temporal medida

Resultado: 67 pasan, 0 fallan
govpadmin@govp-srv:~/proyecto-low-monitor$ ./bin/lowmonitor
LowMonitor v0.7.0 - Diagnóstico y monitorización de hardware x86-64
== [M1] Identificación del CPU ==
Fabricante:       AuthenticAMD
Procesador:       AMD EPYC 7763 64-Core Processor
Arquitectura:     x86_64
Familia:          25 (19h)
Modelo:           1 (01h)
Stepping:         1
Hilos lógicos:    2
Hipervisor:       Microsoft Hv
Características:  RDTSCP, TSC invariante, CLFLUSH, POPCNT
Extensiones:      SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, FMA, AVX2
AVX (SO):         Habilitado (XCR0 guarda YMM)
AVX-512 (SO):     No soportado por la CPU
```

Resultado que entrega: Este módulo nos permite conocer qué procesador estamos utilizando y qué tecnologías admite para ejecutar las diferentes pruebas de rendimiento.

M2. Monitor de puertos de E/S

El módulo *ports.asm* implementa instrucciones IN/OUT de 8, 16 y 32 bits para los puertos contemplados por la práctica. La decisión ADR-004 describe un mecanismo para intentar obtener permisos con *ioperm* y, si se deniegan, capturar la excepción de protección y reportar el estado como protegido, evitando que el proceso termine abruptamente. El acceso real depende de permisos y del hardware o la máquina virtual; la documentación prevé que ciertos dispositivos legados aparezcan como no presentes.

¿Qué es y que hace? Es el módulo encargado de revisar los permisos y el acceso a los puertos de entrada y salida (E/S) del sistema. Comprueba si el programa tiene acceso a determinados puertos de hardware, como los utilizados por el teclado, paneles de control y otros dispositivos.

¿Cómo lo hace? Intenta consultar los puertos mediante instrucciones de entrada y salida del procesador, como ioperm e iopl, y verifica las restricciones de seguridad del sistema operativo.

```
== [M2] Monitor de puertos E/S ==
Permiso ioperm:        DENEGADO (EPERM) - ejecute con sudo para acceso real
Permiso iopl(3):       DENEGADO (EPERM)
IOPL (RFLAGS[13:12]):  0
Puerto 64h (estado):   [PROTEGIDO] #GP capturado (SIGSEGV)
Puerto 60h (teclado):  [PROTEGIDO] #GP capturado (SIGSEGV)
Puerto 61h (panel):    [PROTEGIDO] #GP capturado (SIGSEGV)
Puerto 3F8h (COM1):    [PROTEGIDO] #GP capturado (SIGSEGV)
Puerto CFCh (PCI 32b): [PROTEGIDO] #GP capturado (SIGSEGV)
```

Resultado que entrega: Este módulo permite comprobar si el sistema autoriza el acceso directo a los puertos de hardware. En este caso, los accesos están restringidos por los permisos del sistema operativo y el entorno virtualizado.

M3. Diagnóstico de memoria

El módulo *memtest.asm* reserva un bloque de memoria de usuario mediante mmap y aplica patrones 0xAA, 0x55 e incremental. Compara los valores leídos con los esperados y cuenta bits discrepantes mediante POPCNT cuando está disponible, con una alternativa por software. El código también incorpora una prueba de inyección que altera bits conocidos para comprobar el detector. La memoria evaluada corresponde al bloque reservado por el proceso, no a una inspección directa de toda la RAM física.

¿Qué es y que hace? Es el módulo destinado a evaluar el comportamiento y rendimiento de la memoria del equipo. Analiza aspectos relacionados con el funcionamiento de la memoria, como el acceso y el comportamiento de las operaciones de lectura y escritura.

¿Cómo lo hace?

Realiza pruebas sobre bloques de memoria utilizando diferentes patrones de datos para comprobar que la información pueda escribirse y leerse correctamente.

```
== [M3] Diagnóstico de memoria ==
Memoria total:      3907 MB detectada
Memoria libre:      2285 MB
Bloque probado:     64 MB (mmap + MAP_POPULATE)
Patrón 0xAA:        PASADA (0 errores)
Patrón 0x55:        PASADA (0 errores)
Patrón incremental: PASADA (0 errores)
Escritura (NT):     24917 MB/s
Verificación:       19374 MB/s
```

Resultado que entrega: Presenta la memoria detectada, la memoria disponible, el tamaño del bloque evaluado, el resultado de cada prueba (aprobada o con errores) y las velocidades de escritura y verificación en MB/s. En este escenario todas las pruebas fueron aprobadas con 0 errores.

M4. Rendimiento del CPU

El módulo *perf.asm* calibra el contador TSC, realiza mediciones repetidas y estima métricas de ciclos, IPC, CPI, frecuencia y rendimiento de punto flotante. Incluye cargas con dependencias y cargas independientes para observar la diferencia entre latencia y paralelismo a nivel de instrucción. Cuando los permisos del kernel lo permiten, intenta usar contadores PMU a través de perf_event_open; de lo contrario, el comportamiento queda sujeto a las métricas basadas en TSC y a las condiciones del entorno.

¿Qué es? Es una función que permite evaluar el rendimiento del procesador (CPU), midiendo su capacidad para ejecutar instrucciones, procesar operaciones y realizar cálculos matemáticos.

¿Qué hace y cómo lo hace? Ejecuta diferentes cargas de trabajo sobre el procesador y contabiliza los ciclos necesarios para completarlas.

```
== [M4] Monitor de rendimiento del CPU ==
CPU fijada:           0 (sched_setaffinity)
Frecuencia (TSC):     2.44 GHz
Costo de medición:    49 ciclos
Carga dependiente:    100000000 instr, 60916677 ciclos, CPI 0.60, IPC 1.64
Carga independiente:  100000000 instr, 17148285 ciclos, CPI 0.17, IPC 5.83
Carga FP (SSE2):      100000000 instr, 22948611 ciclos, CPI 0.22, IPC 4.35, 80000000 FLOP
CPI promedio:         0.33 (estimado con TSC)
IPC:                  2.96 (estimado con TSC)
Throughput:           8.52 GFLOPS (SSE2 escalar)
Contadores PMU:       no disponibles (EACCES)
Ciclos totales:       101013573
```

Resultado que entrega: Presenta métricas de rendimiento del procesador, incluyendo frecuencia, ciclos consumidos, instrucciones ejecutadas, eficiencia de procesamiento (CPI e IPC) y capacidad de cálculo matemático (GFLOPS). Resultado observado en este escenario; throughput de 8.52 GFLOPS, IPC promedio de 2.96 y aproximadamente 101 millones de ciclos totales.

Módulos restantes, validación y resultados

M5. Jerarquía de memoria

El módulo *cache.asm* obtiene tamaños de caché mediante CPUID cuando la CPU lo permite y construye recorridos de listas enlazadas con nodos de 64 bytes. Las cargas dependientes y el orden aleatorio buscan reducir la anticipación del prefetcher. Se registran mediciones por acceso y por cadena de cargas; el código advierte que el costo de leer el TSC en cada acceso hace más ruidosa la primera medida. Los resultados pueden variar por procesador, caché compartida, virtualización y frecuencia.

¿Qué es y que hace? Es el módulo que analiza los diferentes niveles de memoria utilizados por el procesador: registros, caché L1, L2, L3 y memoria RAM. Mide cuánto tiempo necesita el procesador para acceder a cada nivel de memoria y compara sus velocidades.

¿Cómo lo hace? Realiza pruebas de acceso a memoria y mide los ciclos del procesador (TSC). También ejecuta pruebas de acceso aleatorio y secuencial para comparar el comportamiento de la memoria.

```
== [M5] Gestión jerárquica de memoria ==
Nivel        Tamaño      Por acceso   Cadena   (ciclos TSC)
Registros    -           16.7         0.7
L1 caché     32 KB       18.0         3.0
L2 caché     512 KB      29.1         11.9
L3 caché     32 MB       51.6         33.5
RAM          3907 MB     227.1        214.0
RAM / L1:    71x
Conjuntos de trabajo: L1 16 KB, L2 272 KB, L3 2 MB, RAM 128 MB

Prefetch sobre RAM (ciclos por línea de 64 B):
  Aleatorio (cadena):       214.0
  Secuencial:               4.6
  Secuencial + prefetcht0:  5.3
```

Resultado que entrega: Este módulo demuestra la importancia de la jerarquía de memoria.

Cuanto más cerca está la memoria del procesador, generalmente menor es el tiempo necesario para acceder a los datos.

M6. Operaciones SIMD

El módulo *simd.asm* implementa suma, multiplicación y comparación de arreglos de números de punto flotante mediante una referencia escalar y rutas SSE2, AVX2 y AVX-512, cuando están disponibles. Compara resultados vectoriales con la referencia y mide un benchmark de operaciones aritméticas, además de comparar escrituras normales y non-temporal. El despacho verifica las capacidades de CPU y sistema operativo antes de ejecutar instrucciones opcionales.

¿Qué es y que hace? SIMD significa Single Instruction, Multiple Data (una instrucción, múltiples datos). Es una tecnología que permite procesar varios datos utilizando una misma instrucción del procesador. Evalúa la capacidad del procesador para realizar operaciones matemáticas en paralelo y compara su rendimiento utilizando diferentes tecnologías.

¿Cómo lo hace? Ejecuta operaciones de suma, multiplicación y comparación utilizando diferentes conjuntos de instrucciones.

```
== [M6] Operaciones SIMD ==
Escalar (referencia):     Sí   3.16 GFLOPS  x1.0
SSE2 (128b, 4 lanes):     Sí   suma [OK]  mult [OK]  comp [OK]  14.44 GFLOPS  x4.5
AVX2 (256b, 8 lanes):     Sí   suma [OK]  mult [OK]  comp [OK]  29.20 GFLOPS  x9.2
AVX-512 (512b, 16 lanes): No soportado por la CPU
FMA (256b):               Sí   29.23 GFLOPS  x9.2
AVX2 habilitado:          Sí
Throughput:               8 operaciones paralelas (AVX2)
Benchmark:                29.23 GFLOPS (FMA (256b))
Escritura normal 64 MB:   19419 MB/s
```

Resultado que entrega: Este módulo demuestra cómo el procesador puede aumentar su rendimiento al ejecutar operaciones matemáticas en paralelo, reduciendo el tiempo necesario para procesar grandes cantidades de información.

Compilación y pruebas automatizadas

El Makefile define los objetivos *make*, *make test*, *make info* y *make evidencia*. El script *tests/smoke.sh* comprueba formato ELF64, arquitectura x86-64, ausencia de intérprete dinámico, selección de módulos por argumentos, formato de salida y pruebas específicas de CPU, E/S, memoria, rendimiento, caché y SIMD. El flujo de GitHub Actions configura un runner Ubuntu 24.04 x86-64, instala NASM, compila sin advertencias, ejecuta las pruebas y publica el binario como artefacto. El flujo de releases recompila y prueba al publicar una etiqueta de versión.

Resultados verificables La inspección del repositorio confirma que existen los seis módulos, el Makefile, las pruebas de humo y los flujos de integración continua. A continuación, se presenta la evidencia de la ejecución de la aplicación LowMonitor.

Conclusiones

LowMonitor presenta una implementación modular de ensamblador x86-64 que aborda las seis áreas de la práctica y añade automatización de compilación, pruebas y publicación. Entre sus decisiones destacadas están el uso exclusivo de syscalls, el enlace estático, el manejo controlado de intentos de acceso a puertos y la comprobación del soporte de instrucciones SIMD. La documentación y los tests muestran un plan de validación estructurado.

Referencias

Equipo de desarrollo de LowMonitor. (2026). *LowMonitor: diagnóstico y monitorización de hardware* [Código fuente y documentación del proyecto]. GitHub.

https://github.com/yesferfuetes/proyecto-low-monitor/tree/main

Intel Corporation. (2025). *Intel® 64 and IA-32 architectures software developer's manual*. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

NASM Development Team. (2024). *NASM—The Netwide Assembler documentation*. https://www.nasm.us/doc/